

Lecture 22: Autoencoders

A network that learns to copy - and why the copy is the point

Parts of this chapter adapt LMU I2DL (slds-lmu), CC BY 4.0.

From 784 numbers to 2, and back

One MNIST digit is a vector of **784** numbers (28×28 pixels).

Predict: can a network squeeze that digit down to just **2 numbers** and then rebuild it - with *no labels*, ever?

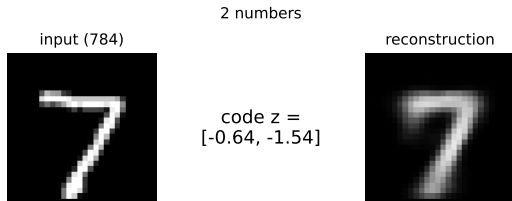
From 784 numbers to 2, and back

One MNIST digit is a vector of **784** numbers (28×28 pixels).

Predict: can a network squeeze that digit down to just **2 numbers** and then rebuild it - with *no labels*, ever?

Basically yes (right). The trick: the network's **target is its own input**. No human labels - it **supervises itself**.

Chapters 5-7 (nets, CNNs, RNNs) were *supervised*. Autoencoders take us back to **unsupervised** learning - the territory of clustering (ch4) and PCA (ch4b) - now with a neural net and no y .



Outline

The autoencoder idea

Autoencoders = nonlinear PCA

Regularized autoencoders

Anomaly detection

Autoencoders in the wild

The autoencoder idea

Learn to rebuild your input, and let the bottleneck do the teaching.

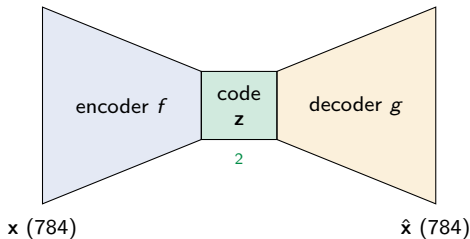
Encoder, code, decoder

An **autoencoder** is two networks trained together:

- **Encoder** $f : \mathbf{x} \mapsto \mathbf{z}$ - compress the input to a small *code* $\mathbf{z}$.
- **Decoder** $g : \mathbf{z} \mapsto \hat{\mathbf{x}}$ - rebuild the input from the code.

Train so that $\hat{\mathbf{x}} \approx \mathbf{x}$. No labels: the target *is* the input.

This is **self-supervised** learning: a supervised loss (rebuild $\mathbf{x}$) with a free label (the input itself).



The bottleneck is the whole point

Make the code **smaller** than the input:

$\dim(\mathbf{z}) < \dim(\mathbf{x})$ (**undercomplete**). The network cannot pass everything through, so it must keep only what matters - the structure the digits share.

No bottleneck, no learning. If $\dim(\mathbf{z}) \geq \dim(\mathbf{x})$ the network can just learn the **identity** - a perfect, useless copy that memorized nothing.

The **loss** is nothing new: squared error $\|\mathbf{x} - \hat{\mathbf{x}}\|^2$ (or cross-entropy for $[0, 1]$ pixels). *Callback*: the L2 loss from regression.

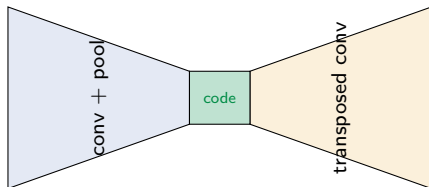


You already built the decoder

For images, the two halves are just a **CNN** and its mirror - a **convolutional autoencoder**:

- ▶ Encoder = convolution + pooling (*ch6, L16*) - shrink to a code.
- ▶ Decoder = **transposed convolution** (*L19*) - grow the code back to an image.

Nothing new to learn here: you built both halves in the CNN chapter. An autoencoder just points them at each other.



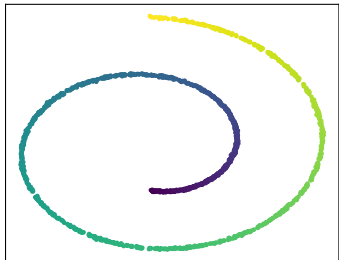
Autoencoders = nonlinear PCA

You already compressed digits once - PCA. This bends the axes.

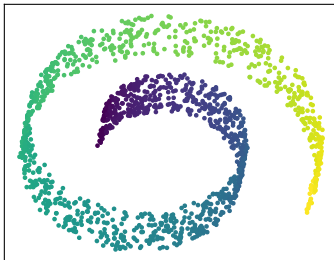
The bridge: a linear autoencoder is PCA

Give the autoencoder **linear** activations and a squared-error loss, and it recovers the same top- k subspace as **PCA** (*ch4b*) - the best **flat sheet** through the data. Add nonlinearity and the sheet can **bend** around curved data a flat one misses.

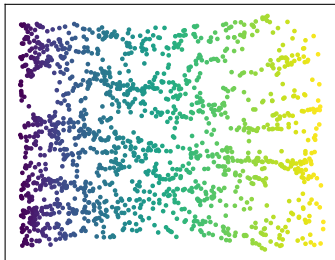
data: a rolled-up sheet



PCA (linear): flat projection
the roll overlaps itself



nonlinear: unrolled
(Isomap here; an AE does the same)

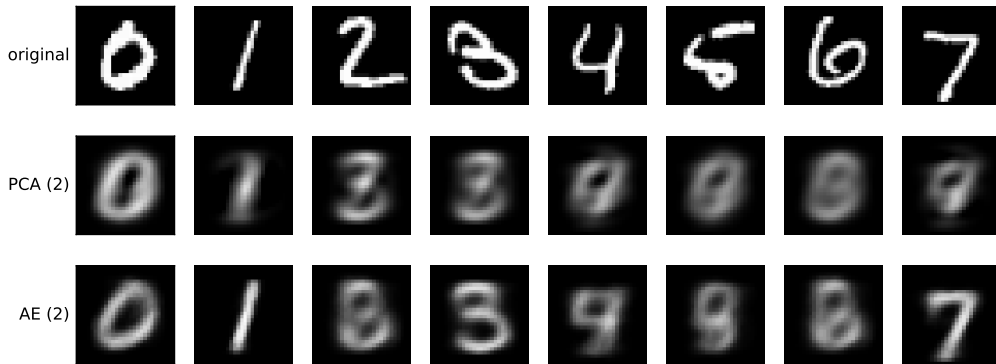


On a curved sheet, PCA's flat projection makes far-apart points **overlap**; a nonlinear method (an autoencoder, or Isomap here) **unrolls** it. That is the whole difference.

Two numbers: what survives, linear vs nonlinear

Compress each test digit to **2 numbers**, then reconstruct. Same budget, two methods:

Reconstruct each digit from 2 numbers: PCA (linear) vs AE (nonlinear)

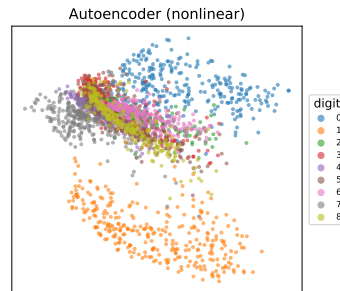
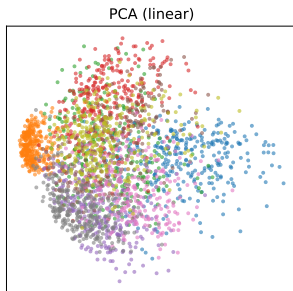


PCA's 2-component reconstruction (linear) collapses several digits to a ghostly *average*; the autoencoder's 2-number code (nonlinear) keeps more real digit structure. Neither is perfect - 2 numbers is brutal - but the curve wins.

The same digits as a 2-D map

Both methods also give a **2-D map** of the data (each point is one digit).

Honest caveat: the AE's axes are **not** orthogonal, **not** variance-ordered, and **not** unique (another training run gives a different map). PCA hands you all of those for free. The AE trades them for flexibility.



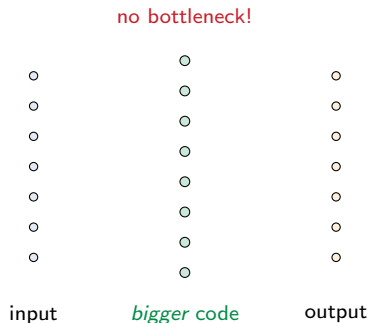
Regularized autoencoders

When you cannot rely on a tight bottleneck, constrain the code another way.

The overcomplete problem

Sometimes you *want* a big code ($\dim(\mathbf{z}) \geq \dim(\mathbf{x})$)
- to learn a rich feature set, not just to compress.
But then the undercomplete trick is gone: the network can copy the input straight through again.

Fix: keep the network honest with a **different constraint**. Two classics - **denoising** and **sparse**.



Denoising autoencoders

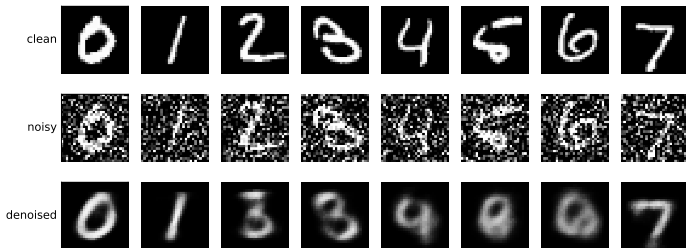
Break the input on purpose.

Corrupt x (add noise, or zero out pixels), then train the network to output the **clean** original.

It cannot copy noise, so it must learn what a digit really is.

Callback: ch5 data augmentation - robustness by perturbation. Here the perturbation is the whole training signal.

Predict: zero out half the pixels and demand the clean digit back - does that really help? Yes.



Sparse autoencoders

Another way to stay honest: don't shrink the code, **penalize its activations**. Add an L_1 penalty (or a target sparsity) so that for any given input, only a **few** code neurons fire.

Callback: the L_1 penalty is Lasso (*ch1*) - same idea, now on hidden activations instead of weights. It forces each input to use a small, specialized set of features.



code neurons
(few fire)

Anomaly detection

If it cannot rebuild you, you do not belong.

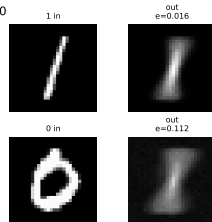
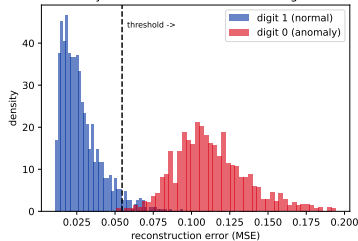
Reconstruction error is an anomaly score

Train an autoencoder on **normal data only**. It learns to rebuild normal inputs with **low** error. Feed it something it has never seen - it rebuilds it **badly**, with **high** error.

So the reconstruction error *is* an anomaly score: pick a **threshold**, flag anything above it.

The industrial workhorse use of autoencoders: fraud, manufacturing defects, network intrusion. *Callback*: thresholding a score - ch3 metrics.

AE trained only on 1s: threshold the error to flag the unseen 0



Trained only on 1s; a 0 gets forced toward a 1-shape $\Rightarrow$ high error.

A caveat you can see

Reconstruction-error anomaly detection only works when the anomaly is **genuinely different** from the training data.

Train on digits 0-8 and try to flag a **9**: it fails - a 9 looks like a 4, a 7, a 1, all of which the network saw, so it reconstructs 9s just fine. The clean split on the previous slide needed a truly distinct anomaly (train on 1s, flag 0s). Know what "normal" and "anomalous" mean for *your* data before you trust the error.

Autoencoders in the wild

The same three ideas, at frontier scale.

The latent idea is everywhere (2026)

Every technique in this deck is load-bearing in today's biggest models:

- ▶ **Self-supervised pretraining** = denoising. **BERT** masks words and rebuilds them; **MAE** masks image patches and rebuilds them. Masked modeling is denoising at scale - the recipe behind most foundation models.
- ▶ **LLM interpretability** = sparse. A wide **sparse autoencoder** trained on a model's activations splits them into single-concept features - how Anthropic reads what is inside Claude (*Scaling Monosemanticity*, 2024).
- ▶ **Cheaper inference** = bottleneck. **DeepSeek's** Multi-head Latent Attention compresses the KV cache into a small **latent vector** - about 4% of the usual size - so long contexts fit in memory (DeepSeek-V3, 2024).

And the biggest use of all is the **variational autoencoder** of the next lecture: it is the compression layer inside modern **image generators**.

Recap: autoencoders

- ▶ An autoencoder is an **encoder + decoder** trained to rebuild its input - **self-supervised**, no labels.
- ▶ The **bottleneck** forces it to keep only structure; a linear one recovers **PCA**, a nonlinear one bends around curved data.
- ▶ Cannot rely on a tight bottleneck? Regularize: **denoising** (corrupt and recover) or **sparse** (penalize activations).
- ▶ **Anomaly detection**: high reconstruction error flags what the network never learned.

But it cannot generate

One thing the autoencoder **cannot** do: invent a new digit. To generate, you would draw a random code $\mathbf{z}$ and decode it. But the AE never told its codes to follow any known distribution.

Draw $\mathbf{z} \sim \mathcal{N}(0, I)$ (red) and the codes (blue) are somewhere else entirely: the draws miss almost the whole manifold and decode to blurry blobs. You can **reconstruct**, but you cannot **sample**.

Next (L23): give the latent space no holes, then sample it - **variational autoencoders**.

